

Exercises 34.2

NP Verification, Hamiltonian Problems, co-NP, and Graph Cubes

Background

Core Idea

A language is in **NP** if every YES instance has a polynomial-size certificate that can be checked in polynomial time. A language is in **co-NP** if its complement is in **NP**.

1 Exercise 34.2-1

Question 34.2-1

Consider the language

$$\text{GRAPH-ISOMORPHISM} = \{\langle G_1, G_2 \rangle \mid G_1 \text{ and } G_2 \text{ are isomorphic graphs}\}.$$

Prove that $\text{GRAPH-ISOMORPHISM} \in \mathbf{NP}$ by describing a polynomial-time verifier.

Answer

The certificate is a proposed isomorphism

$$\pi : V(G_1) \rightarrow V(G_2).$$

Verifier

On input $\langle G_1, G_2 \rangle$ and certificate π , the verifier:

1. checks that π is a bijection from $V(G_1)$ to $V(G_2)$;
2. for every pair $u, v \in V(G_1)$, checks

$$(u, v) \in E(G_1) \iff (\pi(u), \pi(v)) \in E(G_2).$$

Using adjacency matrices, each edge lookup takes constant time, and there are only $\mathcal{O}(|V|^2)$ vertex pairs. Therefore the verifier runs in polynomial time. A valid certificate exists exactly when the graphs are isomorphic. Hence

$$\text{GRAPH-ISOMORPHISM} \in \mathbf{NP}.$$

2 Exercise 34.2-2

Question 34.2-2

Prove that if G is an undirected bipartite graph with an odd number of vertices, then G is nonhamiltonian.

Answer

Every cycle in a bipartite graph has even length. A Hamiltonian cycle would have length $|V|$, so if $|V|$ is odd, no Hamiltonian cycle can exist.

Proof

Let the bipartition be

$$V = S \cup T, \quad S \cap T = \emptyset.$$

Every edge goes between S and T . Therefore any cycle must alternate between vertices in S and vertices in T .

If a cycle starts in S , then after an odd number of edges it is in T , and after an even number of edges it is back in S . To return to its starting vertex, the cycle must have even length.

A Hamiltonian cycle visits every vertex exactly once, so its length is $|V|$. If $|V|$ is odd, this is impossible. Thus G is nonhamiltonian.

3 Exercise 34.2-3**Question 34.2-3**

Show that if HAM-CYCLE $\in \mathbf{P}$, then the problem of listing the vertices of a Hamiltonian cycle, in order, is polynomial-time solvable.

Answer

Assume we have a polynomial-time decider D for HAM-CYCLE. We use D as a subroutine to delete unnecessary edges until only one Hamiltonian cycle remains.

Algorithm

Given a graph G :

1. Run $D(G)$. If $D(G) = \text{No}$, report that no Hamiltonian cycle exists.
2. Set $H \leftarrow G$.
3. For each edge $e \in E(G)$:
 - a. Run $D(H - e)$.
 - b. If $H - e$ is Hamiltonian, set $H \leftarrow H - e$.
 - c. Otherwise, keep e .
4. Traverse the remaining graph H to list the cycle vertices in order.

Correctness

The algorithm maintains the invariant that H is Hamiltonian. Whenever an edge can be removed while preserving Hamiltonicity, we remove it.

At the end, let C be any Hamiltonian cycle in H . If some remaining edge $e \in E(H)$ were not in C , then $H - e$ would still contain C and would still be Hamiltonian. When the algorithm considered e , the graph at that time contained the final graph H , so deleting e would also have left a Hamiltonian graph. The algorithm would have removed e , a contradiction.

Therefore every remaining edge lies on the Hamiltonian cycle C , so H is exactly a Hamiltonian cycle. Traversing H lists the vertices in order.

Running Time

The algorithm makes at most $|E| + 1$ calls to the polynomial-time decider and does only polynomial graph operations. Therefore it runs in polynomial time.

4 Exercise 34.2-4**Question 34.2-4**

Prove that the class **NP** of languages is closed under union, intersection, concatenation, and Kleene star. Discuss closure of **NP** under complement.

Answer

Let $L_1, L_2 \in \mathbf{NP}$, with polynomial-time verifiers A_1, A_2 and polynomial-size certificates.

Union

For $L_1 \cup L_2$, the certificate is

$$(b, y),$$

where $b \in \{1, 2\}$ indicates which language to verify. If $b = 1$, run $A_1(x, y)$; if $b = 2$, run $A_2(x, y)$. Accept if the selected verifier accepts. Thus $L_1 \cup L_2 \in \mathbf{NP}$.

Intersection

For $L_1 \cap L_2$, the certificate is a pair

$$(y_1, y_2),$$

where y_1 verifies $x \in L_1$ and y_2 verifies $x \in L_2$. Run both verifiers and accept if both accept. Thus $L_1 \cap L_2 \in \mathbf{NP}$.

Concatenation

For $L_1 L_2$, a certificate consists of a split position i and certificates y_1, y_2 such that

$$x = x_1 x_2, \quad |x_1| = i, \quad x_1 \in L_1, \quad x_2 \in L_2.$$

The verifier checks the split and runs $A_1(x_1, y_1)$ and $A_2(x_2, y_2)$. This is polynomial time, so $L_1 L_2 \in \mathbf{NP}$.

Kleene Star

For L^* , a certificate gives:

- split positions decomposing x into nonempty pieces

$$x = x_1 x_2 \cdots x_t;$$

- certificates y_i proving $x_i \in L$ for each i .

If x is empty, the verifier accepts using the empty decomposition. Otherwise $t \leq |x|$. If certificates for L have length at most $p(n)$ for a polynomial p , then the total certificate length is at most

$$|x| \cdot p(|x|),$$

which is polynomial.

The verifier checks every piece and every certificate in polynomial time. Hence $L^* \in \mathbf{NP}$.

Complement

It is not known whether **NP** is closed under complement. A failed certificate for $x \in L$ does not prove $x \notin L$; it only proves that this particular certificate did not work. The question of complement closure is exactly whether

$$\mathbf{NP} = \mathbf{co-NP}.$$

5 Exercise 34.2-5**Question 34.2-5**

Show that any language in **NP** can be decided by an algorithm running in time

$$2^{\mathcal{O}(n^k)}$$

for some constant k .

Answer

An **NP** verifier has polynomial-size certificates. A deterministic algorithm can enumerate all possible certificates and run the verifier on each one.

Proof

Let $L \in \mathbf{NP}$. Then there is a polynomial-time verifier $A(x, y)$ and a polynomial $p(n) = \mathcal{O}(n^c)$ such that every useful certificate has length at most $p(|x|)$.

For input x of length n , enumerate all binary strings y of length at most $p(n)$. There are

$$2^{\mathcal{O}(n^c)}$$

such strings. For each one, run $A(x, y)$. If any run accepts, accept; otherwise reject.

The verifier's polynomial running time is absorbed into the exponential bound. Thus the total running time is

$$2^{\mathcal{O}(n^k)}$$

for some constant k .

6 Exercise 34.2-6**Question 34.2-6**

A Hamiltonian path in a graph is a simple path that visits every vertex exactly once. Show that

$$\text{HAM-PATH} = \{\langle G, u, v \rangle \mid \text{there is a Hamiltonian path from } u \text{ to } v \text{ in } G\}$$

belongs to **NP**.

Answer

The certificate is the proposed path:

$$(v_1, v_2, \dots, v_n).$$

Verifier

The verifier checks:

1. $v_1 = u$ and $v_n = v$;
2. the list is a permutation of all vertices in G ;
3. for each $i = 1, 2, \dots, n - 1$, the edge (v_i, v_{i+1}) exists.

All checks are polynomial using an adjacency matrix or adjacency lists. Hence

$$\text{HAM-PATH} \in \mathbf{NP}.$$

7 Exercise 34.2-7**Question 34.2-7**

Show that the Hamiltonian-path problem from Exercise 34.2-6 can be solved in polynomial time on directed acyclic graphs. Give an efficient algorithm.

Answer

In a directed acyclic graph, any directed Hamiltonian path must appear in topological order. Therefore we compute a topological ordering and check whether consecutive vertices are connected.

Algorithm

Given a DAG $G = (V, E)$ and vertices u, v :

1. Compute a topological ordering

$$v_1, v_2, \dots, v_n.$$

2. If $v_1 \neq u$ or $v_n \neq v$, reject.
3. For each $i = 1, 2, \dots, n - 1$, check whether

$$(v_i, v_{i+1}) \in E.$$

4. Accept if all these edges exist; otherwise reject.

Correctness

If the algorithm accepts, then the edges

$$(v_1, v_2), (v_2, v_3), \dots, (v_{n-1}, v_n)$$

form a directed path visiting every vertex exactly once, starting at u and ending at v .

Conversely, suppose there is a Hamiltonian path from u to v :

$$u = w_1 \rightarrow w_2 \rightarrow \dots \rightarrow w_n = v.$$

In a DAG, if there is a directed path from a to b , every topological order places a before b . Therefore every topological order must list the vertices as

$$w_1, w_2, \dots, w_n.$$

Thus consecutive vertices in the computed topological order must be connected by the Hamiltonian-path edges, and the algorithm accepts.

Running Time

Topological sorting takes $\mathcal{O}(|V| + |E|)$ time. Checking consecutive edges is polynomial, and with adjacency matrices it takes $\mathcal{O}(|V|)$ time after preprocessing.

8 Exercise 34.2-8**Question 34.2-8**

Let ϕ be a Boolean formula built from variables, negations, ANDs, ORs, and parentheses. The formula is a tautology if it evaluates to 1 for every truth assignment. Define TAUTOLOGY as the language of tautological formulas. Show that

$$\text{TAUTOLOGY} \in \mathbf{co-NP}.$$

Answer

To prove TAUTOLOGY is in **co-NP**, prove that its complement is in **NP**.

Proof

The complement is

$$\overline{\text{TAUTOLOGY}} = \{\phi \mid \phi \text{ is not a tautology}\}.$$

A formula is not a tautology if there exists a truth assignment making it false.

Thus a certificate for $\overline{\text{TAUTOLOGY}}$ is a truth assignment to the variables. The verifier evaluates ϕ under this assignment and accepts if the value is 0.

Formula evaluation takes polynomial time in the length of ϕ . Therefore

$$\overline{\text{TAUTOLOGY}} \in \mathbf{NP}.$$

By definition of **co-NP**,

$$\text{TAUTOLOGY} \in \mathbf{co-NP}.$$

9 Exercise 34.2-9**Question 34.2-9**

Prove that

$$\mathbf{P} \subseteq \mathbf{co-NP}.$$

Answer

If a language is decidable in polynomial time, then nonmembership is also decidable in polynomial time by flipping the answer.

Proof

Let $L \in \mathbf{P}$. Then there is a polynomial-time decider A for L . Construct a verifier for $\bar{L}$ that ignores its certificate, runs $A(x)$, and accepts exactly when $A(x)$ rejects.

Thus $\bar{L} \in \mathbf{NP}$, and by definition this means

$$L \in \mathbf{co-NP}.$$

Therefore

$$\mathbf{P} \subseteq \mathbf{co-NP}.$$

10 Exercise 34.2-10

Question 34.2-10

Prove that if

$$\mathbf{NP} \neq \mathbf{co-NP},$$

then

$$\mathbf{P} \neq \mathbf{NP}.$$

Answer

Use the contrapositive: if $\mathbf{P} = \mathbf{NP}$, then $\mathbf{NP} = \mathbf{co-NP}$.

Proof

Assume $\mathbf{P} = \mathbf{NP}$. Since $\mathbf{P}$ is closed under complement, every language in $\mathbf{P}$ has its complement in $\mathbf{P}$.

Now let $L \in \mathbf{NP}$. Because $\mathbf{P} = \mathbf{NP}$, we have $L \in \mathbf{P}$. Therefore $\bar{L} \in \mathbf{P}$. Since $\mathbf{P} \subseteq \mathbf{NP}$, it follows that $\bar{L} \in \mathbf{NP}$.

Thus every language in $\mathbf{NP}$ has its complement in $\mathbf{NP}$, so

$$\mathbf{NP} = \mathbf{co-NP}.$$

The contrapositive gives:

$$\mathbf{NP} \neq \mathbf{co-NP} \implies \mathbf{P} \neq \mathbf{NP}.$$

11 Exercise 34.2-11

Question 34.2-11

Let G be a connected, undirected graph with at least 3 vertices, and let G^3 be the graph obtained by connecting all pairs of vertices that are connected by a path in G of length at most 3. Prove that G^3 is Hamiltonian.

Answer

It is enough to prove the result for trees. Choose a spanning tree T of G . If T^3 has a Hamiltonian cycle, then G^3 also has one because

$$T^3 \subseteq G^3.$$

The Tree Case

DFS Construction

Root the tree T at any vertex r . Perform a depth-first traversal starting and ending at r . During the traversal, record each vertex exactly once using this rule:

- record a vertex at odd depth when it is first visited;
- record a vertex at even depth when it is last visited.

This produces a list

$$v_1, v_2, \dots, v_n$$

containing every vertex of T exactly once.

Why Consecutive Vertices Are Close

Consider two consecutive recorded vertices in the DFS list. Since no other vertex is recorded between them, the DFS walk cannot pass through a long unrecorded portion of the tree.

If the first recorded vertex x has odd depth, it was recorded when first entered. After this moment, the next recorded vertex occurs in one of the following ways:

- if x has an even-depth child that is a leaf, that child is recorded when finished, at distance 1 from x ;
- if that child has an odd-depth child, the odd-depth child is recorded when first entered, at distance 2 from x ;
- if x has no unvisited child, the traversal returns through the parent or moves to a sibling subtree, giving the next record within distance at most 2.

If the first recorded vertex x has even depth, it was recorded when its subtree was finished. The traversal then moves to its odd-depth parent. From there, the next record is either:

- the even-depth grandparent, at distance 2 from x ;
- an even-depth sibling subtree root, at distance 2 from x ;
- or an odd-depth child inside that sibling subtree, at distance 3 from x .

Thus, for every $i = 1, 2, \dots, n-1$,

$$\text{dist}_T(v_i, v_{i+1}) \leq 3.$$

The last recorded vertex is the root r , and the first recorded vertex is a child of r , so

$$\text{dist}_T(v_n, v_1) \leq 1 \leq 3.$$

Conclusion

Since consecutive vertices in the cyclic order

$$v_1, v_2, \dots, v_n, v_1$$

are at distance at most 3 in T , every corresponding edge exists in T^3 . These edges form a Hamiltonian cycle in T^3 .

Because $T^3 \subseteq G^3$, the same cycle exists in G^3 . Hence G^3 is Hamiltonian.

Final Summary

What to Remember

- Graph isomorphism is in **NP** because a bijection of vertices is a polynomially checkable certificate.
- Bipartite graphs have only even cycles, so an odd-order bipartite graph cannot be Hamiltonian.
- A Hamiltonian-cycle decider can be used to construct an actual cycle by deleting unnecessary edges.
- **NP** is closed under union, intersection, concatenation, and Kleene star; complement closure is the open question **NP** = **co-NP**.
- Every **NP** language can be decided by brute-force certificate search in exponential time.
- The cube of every connected graph with at least three vertices is Hamiltonian.